

ECE/CS 552 Introduction to Computer Architecture
Fall 2023
University of Wisconsin-Madison
Prof. Joshua San Miguel

Final Exam
Friday, December 15, 2023
120 Minutes, Wendt Commons 324

1. The exam is two-sided and has 16 pages, which includes two blank pages and the two-page MIPS Green Card in the end.
2. Closed book/closed notes examination.
3. No handheld computers or portable computers allowed.
4. Calculator allowed.
5. A single 8.5"x11" double-sided, hand-written cheat sheet is allowed.
6. When applicable, show your work. Any partial points will be awarded based only on what appears in your answers.
7. No one should leave the room during the last 5 minutes of the examination.
8. Upon announcement of the end of the exam, stop writing on the exam paper immediately. Wait for proctors to pick up all exams. The instructor will announce when you may leave the room.
9. Failure to follow instructions may result in forfeiture of your exam and will be handled according to UWS 14 academic misconduct procedures.

FIRST (GIVEN) NAME: _____

LAST (FAMILY) NAME: _____ **SOLUTION** _____

STUDENT ID#: _____

	Achieved Marks	Maximum Marks
Problem 1		/ 18
Problem 2		/ 6
Problem 3		/ 6
Problem 4		/ 19
Problem 5		/ 5
Problem 6		/ 7
Problem 7		/ 10
Total		/ 71

Problem 1 (18 marks)

(a) (3 marks) If you run a program with a **cache** that is fully-associative, initially empty and has **infinite** capacity, then which of the following types of **cache misses** are **possible**? Select all that apply.

- i. Cold (compulsory) misses
- ii. Capacity misses
- iii. Conflict misses

Only i

(b) (3 marks) You plan to build a **virtually-indexed, physically-tagged** (VIPT) cache but would like to avoid the **synonym problem**. Without changing the cache capacity nor the page size, would it be better to decrease the **associativity** of your cache or increase it? Explain your answer.

Increasing associativity is better, because it reduces the number of bits needed for the set index, which makes the set index more likely to fit within the page offset bits. With maximum associativity (i.e., fully-associative), the synonym problem is completely avoided.

(c) (4 marks) Consider your program running on a **parallel** processor. If your program is 90% parallelizable, what would be the **speedup** of your entire program when executed on a multiprocessor with **infinitely** many cores?

$1 / (0.1 + 0.9 / \infty) = 10x \text{ speedup}$

- (d) (4 marks) Assume a **shared-memory multiprocessor** with two private caches (Cache A and B) using the **modified-shared-invalid (MSI) coherence protocol** from class. In the table below, the rows indicate the possible states of Cache A and the columns indicate the possible states of Cache B for a single shared memory block X. In each cell in the table, specify if it is **possible** (Y/N) for the two caches to be in their respective states (for block X) **at the same time**. One cell has been filled in for you.

Current State of Block X		Cache B		
		I	S	M
Cache A	I	Y	Y	Y
	S	Y	Y	N
	M	Y	N	N

- (e) (4 marks) You use **Booth's algorithm** to perform 8-bit signed integer **multiplication**. If the multiplicand is -47 (in decimal) and the multiplier is -9 (in decimal), answer the following:

i. How many addition operations are needed?

ii. How many subtraction operations are needed?

Booth's encoding of multiplier (11110111₂) = 0 0 0 - + 0 0 -
Thus 1 addition operation and 2 subtraction operations

Problem 2 (6 marks)

Consider a (little-endian) MIPS system with 32-bit virtual addresses, 24-bit physical addresses and a page size of 4KB. The processor has an 8-entry 2-way set-associative **TLB** and a 2-way set-associative **data cache** with 32B capacity and 8B block size, the contents of which are shown below. The cache is **physically-indexed physically-tagged** (PIPT). Both the TLB and cache use the standard set indexing scheme described in class. You run the following instruction:

```
lbu $s0, 0($s1)      # [$s1] = [0x4C00900C]
```

TLB

Set	18-bit Tag	12-bit PPN
0	0x0F001	0x900
	0x13002	0xCC0
1	0x2C100	0x660
	0x13002	0xB2E
2	0x33004	0x2C0
	0x0F001	0xCC4
3	0x2C002	0x66C
	0x0F000	0x902

Data Cache

Set	20-bit Tag	Data Words in Block	
		Offset 0	Offset 4
0	0x660CC	0x11111111	0xAAAAAAAA
	0xB2E00	0x22222222	0BBBBBBBBB
1	0xB2E00	0x33333333	0xCCCCCCCC
	0xCC000	0x44444444	0xDDDDDDDD

(a) (2 marks) What are the **set** and **tag** bits (in hexadecimal) when accessing the **TLB**?

Set = 1, Tag = 0x13002 → physical address = 0xB2E00C

(b) (3 marks) What are the **set**, **offset** and **tag** bits (in hex) when accessing the **cache**?

Set = 1, Tag = 0xB2E00, Offset = 4

(c) (1 mark) What are the **32-bit contents of \$s0** (in hex) after you run the above instruction?

[\$s0] = 0x000000CC

Problem 3 (6 marks)

You have a processor system with 32-bit virtual addresses and a **page size** of 64KB; thus **virtual page numbers** (VPNs) are 16-bit. You have been running a program on this system, and it has accessed memory in the code, heap and stack segments as below. Assume all virtual addresses within the ranges shown are currently residing in physical memory (and no virtual pages outside these ranges have been allocated).

Code segment: 0x00000000 to 0x00002000

Heap segment: 0x30000000 to 0x40020B00

Stack segment: 0x7FFD0000 to 0x7FFFFFFF

The system employs a **two-level page table** that is accessed by dividing the 16-bit VPN into two fields: the most-significant 8 bits are used to index into the **first-level** table, and the least-significant 8 bits are used to index into the **second-level** tables.

- (a) (3 marks) How many entries are currently **valid** (i.e., not a null pointer) in the **first-level** page table? Show your work.

Number of entries for code = $\text{count}\{ 0x00 \} = 1$

Number of entries for heap = $\text{count}\{ 0x30, \dots, 0x40 \} = 17$

Number of entries for stack = $\text{count}\{ 0x7F \} = 1$

Thus 19 total entries

- (b) (3 marks) How many entries are currently **valid** (i.e., a physical page number exists) in total across **all** the **second-level** page tables? Show your work.

Number of entries for code = $\text{count}\{ 0x0000 \} = 1$

Number of entries for heap = $\text{count}\{ 0x3000, \dots, 0x4002 \} = 4099$

Number of entries for stack = $\text{count}\{ 0x7FFD, 0x7FFE, 0x7FFF \} = 3$

Thus 4103 total entries

Problem 4 (19 marks)

Consider the following MIPS assembly program running on the **R10K-like processor** from class, with 15 physical registers:

```

L0:  lw    $t2, 0($t1)
L1:  ori    $t1, $t6, $t2
L2:  sub    $t5, $t1, $t3
L3:  addi   $t3, $t4, $t6
L4:  lw    $t4, 4($t3)
L5:  sw     $t6, 0($t3)
L6:  add    $t1, $t1, $t5
L7:  sub    $t6, $t2, $t5

```

Assume that **initially**: the reservation stations and reorder buffer are empty, and the contents of the map table and free list are as follows. Registers are popped from the left of the free list and pushed back onto the right.

Map Table		
Architectural Reg	Physical Reg	Ready (Y/N)
\$t1	P1	Y
\$t2	P2	Y
\$t3	P3	Y
\$t4	P4	Y
\$t5	P5	Y
\$t6	P6	Y

Free List	P7, P8, P9, P10, P11, P12, P13, P14, P15
------------------	--

All instructions use the **same** type of reservation station and functional unit, and they take 1 cycle to execute (i.e., 1-cycle X stage). Assume that there are **infinitely** many reservation stations and functional units. The pipeline diagram is provided below.

Insn	Cycle															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
L0	F	De	Di	S	X	C	R									
L1		F	De	Di	S	S	X	C	R							
L2			F	De	Di	S	S	S	X	C	R					
L3				F	De	Di	S	X	C	R	R	R				
L4					F	De	Di	S	S	X	C	R	R			
L5						F	De	Di	S	X	C	R	R	R		
L6							F	De	Di	S	X	C	R	R	R	
L7								F	De	Di	Di	S	X	C	R	R

- (a) (12 marks) Specify the contents of the map table **at the end of cycle 9** (the bolded line in the pipeline diagram above).

Map Table		
Architectural Reg	Physical Reg	Ready (Y/N)
\$t1	P12	N
\$t2	P7	Y
\$t3	P10	Y
\$t4	P11	N
\$t5	P9	N
\$t6	P6	Y

- (b) (2 marks) Which **physical register** is causing instruction L2 to **wait** in the Issue stage at cycle 6? In other words, which physical register is L2 waiting for?

P8

- (c) (2 marks) At the **end of which cycle** is physical register P3 **pushed back** onto the free list?

Cycle 12

- (d) (3 marks) Is it possible to know **exactly** how many **reorder buffer entries** there are in this processor? Explain why or why not. And if so, state the number of entries.

Yes, it is possible because instruction L7 stalls at the Dispatch stage in Cycle 10. Since the free list is not empty and there are infinitely many reservation stations, L7 must be stalling due to limited reorder buffer entries. Thus, there are 5 entries in the reorder buffer.

Problem 5 (5 marks)

Consider the following 8-bit **SECEDED** codeword ($W_8W_7W_6W_5W_4W_3W_2W_1$), which encodes a 4-bit data value ($b_4b_3b_2b_1$), using the encoding scheme described in class (shown below):

Codeword Bits							
W_8	W_7	W_6	W_5	W_4	W_3	W_2	W_1
1	0	0	1	1	0	0	0

$S_3S_2S_1, S_p$	Bit flipped?
000, 0	none
001, 1	W_1
010, 1	W_2
011, 1	W_3
100, 1	W_4
101, 1	W_5
110, 1	W_6
111, 1	W_7
000, 1	W_8
non-zero, 0	double bit-flip

Syndrome, Parity	
S_1	$W_1 \oplus W_3 \oplus W_5 \oplus W_7$
S_2	$W_2 \oplus W_3 \oplus W_6 \oplus W_7$
S_3	$W_4 \oplus W_5 \oplus W_6 \oplus W_7$
S_p	$W_8 \oplus \dots \oplus W_1$

Codeword	
W_1	$C_1 = b_1 \oplus b_2 \oplus b_4$
W_2	$C_2 = b_1 \oplus b_3 \oplus b_4$
W_3	b_1
W_4	$C_3 = b_2 \oplus b_3 \oplus b_4$
W_5	b_2
W_6	b_3
W_7	b_4
W_8	$P = W_1 \oplus \dots \oplus W_7$

Compute the **syndrome bits** (S_1, S_2, S_3, S_p) and specify which one of the following has occurred:

- (i) None of the bits in the codeword have flipped.
- (ii) One of the bits in the codeword has flipped.
- (iii) Two different bits in the codeword have flipped.

If either (i) or (ii) has occurred, what was the **original 4-bit data** ($b_4b_3b_2b_1$)?

$$S_1 = W_1 \oplus W_3 \oplus W_5 \oplus W_7 = 1$$

$$S_2 = W_2 \oplus W_3 \oplus W_6 \oplus W_7 = 0$$

$$S_3 = W_4 \oplus W_5 \oplus W_6 \oplus W_7 = 0$$

$$S_p = W_1 \oplus W_2 \oplus W_3 \oplus W_4 \oplus W_5 \oplus W_6 \oplus W_7 \oplus W_8 = 1$$

Therefore, (ii) has occurred: W_1 has flipped.

The original data is 0010.

Problem 6 (7 marks)

Consider the MIPS program below with two threads running in parallel on a **2-core shared-memory multiprocessor**. There are no private caches, and all memory contents are **initially 0**.

# thread A	# thread B
A0: ori \$s0, \$zero, 4000	B0: ori \$s0, \$zero, 4000
A1: addi \$t1, \$zero, 1	B1: addi \$t2, \$zero, 1
A2: sw \$t1, 0(\$s0)	B2: sw \$t2, 4(\$s0)
A3: lw \$t1, 4(\$s0)	B3: lw \$t2, 0(\$s0)

(a) (4 marks) Which of the following program outcomes are **possible** assuming the multiprocessor supports **sequential consistency**? Select all that apply.

- i. [\$t1] = 0 and [\$t2] = 0
- ii. [\$t1] = 0 and [\$t2] = 1
- iii. [\$t1] = 1 and [\$t2] = 0
- iv. [\$t1] = 1 and [\$t2] = 1

ii, iii and iv are possible.

(b) (3 marks) What is the **minimum number of memory fences** needed to guarantee sequential consistency for this program when executed on any arbitrary multiprocessor? And where exactly would you **insert** them into the code?

**2 memory fences are needed:
One between A2 and A3 and one between B2 and B3.**

Problem 7 (10 marks)

You build a **custom floating-point representation** that has less exponent and significand bits than the IEEE 754 standard. Your representation has the following simplified 6-bit format:

2-bit Exponent (with Bias of 1)	4-bit Significand (with an implicit leading 1)
---------------------------------	--

Note that the significand is encoded as 4 bits, excluding the leading 1. This representation does not support negative numbers (i.e., no sign bit) nor special values (e.g., zero, infinity). With your representation, every number is simply computed as:

$$1.\text{Significand} \times 2^{(\text{Exponent} - \text{Bias})}$$

- (a) (5 marks) Given this encoding, fill in the table below, specifying **both** the binary and decimal encoding for each of the following numbers.

	Decimal	Binary
Largest Number Representable	7.75	11 1111
Smallest Number Representable	0.5	00 0000
0.75	0.75	00 1000

- (b) (5 marks) Upon implementing your custom floating point representation, you **forget** to support the implicit leading 1. Your representation now encodes the following instead:

$$0.\text{Significand} \times 2^{(\text{Exponent} - \text{Bias})}$$

Given this buggy encoding, fill in the table below, specifying **both** the binary and decimal encoding for each of the following numbers. If there are multiple encodings for the number, you must list **all** of them.

	Decimal	Binary
Largest Number Representable	3.75	11 1111
Smallest Number Representable	0	00 0000 01 0000 10 0000 11 0000
0.75	0.75	01 1100 10 0110 11 0011

This page is left intentionally blank, in case you need extra space for answers.